2C: Bagging, random forests, and diagnostics

Problem statement and worked resolution

Companion to `02_C_forest_diagnostics.ipynb`

How to use this explanation. This resolution distinguishes probability averaging, out-of-bag evaluation, similarity between trees, bootstrap stability, and feature importance. These answer different questions and should not be interpreted interchangeably. Section numbers follow the notebook's numbered sections where present. Numerical values are rounded and refer to the recorded run, not universal performance claims.

Problem and notation

Generate 2,000 rows with 20 features. Features $x_1, x_3, x_5, \ldots, x_{20}$ are independent uniform draws on $(-1, 1)$. Let $x_2 = x_1 + u$, $u \sim N(0, 0.03^2)$, and let x_4 be independent Bernoulli noise with success probability 0.5. Generate labels using

$$\pi(x) = \sigma\big(-0.7 + 2.5x_1 + 2 \cdot \mathbf{1}\{x_3 > 0.2\}\big), \qquad Y \mid x \sim \text{Bernoulli}(\pi(x)).$$

The indicator is one when $x_3 > 0.2$ and zero otherwise. The sigmoid is $\sigma(z) = 1/(1 + e^{-z})$. Only x_1 and x_3 directly enter the generator; x_2 is a useful near-copy of x_1. Other columns carry no true signal.

Let B be the number of trees, m the number of candidate features offered at a split, and $p = 20$ the total feature count. Out-of-bag (OOB) rows are rows not selected for a particular tree's bootstrap sample. Mean decrease in impurity (MDI) is a training-based feature-importance measure. Cross-validation (CV) selects settings using held-out training folds. The questions are how averaging affects prediction, how to audit OOB bookkeeping, and what importance measures mean with redundant and noise features.

1 Generate data and reserve splits

Simulation seed 17 and stratified split seed 23 give 1,200 training, 400 validation, and 400 test rows. Training has 599 positives and 601 negatives, and x_1, x_2 correlation about 0.99854. The baseline predicts training prevalence 599/1200. No learned preprocessing is applied before the bootstrap, so manual OOB interpretation is direct. True probabilities are retained for simulation only and excluded from features.

Main Python calls

`expit` computes sigmoid probabilities; `train_test_split` separates rows. The helper `positive_probability` locates the positive-class column through `classes_`, rather than blindly assuming a second column always exists. A bootstrap tree can in principle see only one class. The helper returns zero if class 1 is absent, and correctly returns one for a model trained solely on class 1. Tiny tests verify both cases. `clipped_log_loss` clips all reported probabilities to $[10^{-12}, 1 - 10^{-12}]$ consistently; this is not fitted probability smoothing.

2 Select candidate-feature count and leaf size

A classification forest averages leaf probabilities:

$$\hat{p}_B(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{p}_b(x).$$

This differs from counting hard majority votes. Each tree receives a bootstrap sample of its fitting rows. At each node, feature randomization offers a subset of columns for split search. With all 20 features available, this notebook's forest is the bagging comparison.

Main Python calls

`RandomForestClassifier` combines bootstrap trees. `n_estimators=300` fixes B; `max_features=m` controls feature sampling; `min_samples_leaf` limits leaf size; `bootstrap=True` enables resampling; `n_jobs=1` uses one worker. `StratifiedKFold` provides five fitting/scoring partitions. Search $m \in \{1, 4, 10, 20\}$ and minimum leaf size in $\{1, 5, 20\}$. Every setting uses 300 trees and the same five training folds: 960 rows fit and 240 score each time. Average held-out Brier scores,

$$\text{Brier} = \frac{1}{r}\sum_{i=1}^{r}(\hat{p}_i - y_i)^2.$$

An exact tie uses loop order, smaller m then smaller leaf size. The selected setting is $m = 10$, leaf size 20, with CV Brier about 0.156185. The best $m = 20$ setting scores about 0.156360, a very small difference. At $m = 1$, leaf size 20, the score is about 0.196188. Randomizing more aggressively does not automatically improve prediction: diversity can come at the cost of weaker individual split choices. The number 300 is prespecified, not chosen from later OOB curves. Refitting selected settings on all training rows occurs before final diagnostics.

3 Fit comparisons and measure tree-rule similarity

Fit a single tree and 300-tree forests at all four m values, using the selected minimum leaf size for every comparison. This matching rule was specified in advance. It is not a separate optimal tuning of the single tree or bagging baseline.

The intuition for averaging can be expressed, under equal variance and pairwise correlation assumptions at a fixed input, as

$$\text{Var}\left(\frac{1}{B}\sum_{b}T_b(x)\right) = \sigma_T^2\left(\rho + \frac{1-\rho}{B}\right).$$

The formula describes variation across randomized training procedures at the same x. Reducing correlation can help, but bias and tree strength also matter.

The notebook's next statistic is *not* an estimate of this ρ. Generate 400 unlabeled diagnostic inputs, retaining the near-copy relationship and binary noise feature. For the first 100 trees, compute each probability vector across these inputs and correlate vector pairs. `estimators_` accesses individual trees; `np.corrcoef` compares their vectors; constant vectors are omitted because correlation is undefined for them.

Candidate features m	Median prediction-vector correlation
1	0.1966
4	0.5838
10	0.7194
20	0.7414

Smaller feature subsets produce less similar response patterns across inputs here. The CV results show why this observation alone does not determine the best predictor. `perf_counter` records local fit times, not statistical quality.

4 Check OOB bookkeeping on four rows

A row's OOB prediction averages only trees that did not train on it. For bootstrap lists $[0, 0, 1, 2]$, $[1, 1, 2, 3]$, and $[0, 2, 2, 3]$, the excluded rows are respectively 3, 0, and 1. The supplied prediction vectors for those trees are $(2, 4, 6, 8)$, $(3, 5, 7, 9)$, and $(4, 6, 8, 10)$. They are generic numerical predictions for the bookkeeping example, not probabilities.

Row	OOB tree count	OOB prediction
0	1	3
1	1	6
2	0	undefined
3	1	8

`np.bincount` counts bootstrap multiplicities; a count of zero marks OOB status. Row 2 has no eligible prediction, so it remains `NaN`; assigning zero would invent a prediction. Coverage, the fraction of rows with at least one OOB prediction, is 3/4.

5 Implement bagging and audit OOB curves independently

Fit 600 trees with all 20 features available and the selected leaf size. Save every bootstrap list, OOB mask, and training prediction vector. For row i, define $M_{bi} = 1$ if tree b excludes it and zero otherwise. Then

$$N_i(B) = \sum_{b=1}^{B} M_{bi}, \qquad \hat{p}_i^{\mathrm{OOB}}(B) = \frac{\sum_{b=1}^{B} M_{bi}\hat{p}_b(x_i)}{N_i(B)}, \quad N_i(B) > 0.$$

This differs from the ordinary forest average, which includes trees trained on row i.

Main Python calls

`rng.integers` generates bootstrap draws; `np.bincount` converts repetitions to sample weights. `DecisionTreeClassifier.fit(..., sample_weight=multiplicity)` fits the weighted tree. Boolean masks accumulate only eligible probabilities. The weighting representation matches the reference forest: zero-weight rows do not contribute to splits or leaf averages. Minimum leaf size counts distinct included rows in this representation, while weights retain repeated draws. Literally duplicating rows before fitting can interact differently with count-based leaf constraints; this implementation deliberately uses multiplicities as weights.

For $n = 1200$, a row is omitted with probability

$$q_n = (1 - 1/n)^n \approx 0.367726,$$

since each of the n draws avoids it with probability $1 - 1/n$. Thus expected OOB tree count is Bq_n, and approximate coverage under independent draws is $1 - (1 - q_n)^B$.

Trees	OOB Brier	Coverage	Mean OOB count
10	0.168660	0.990833	3.702
30	0.160113	1.000000	11.090
100	0.157260	1.000000	36.789
300	0.157443	1.000000	110.213
600	0.157494	1.000000	220.389

At 600 trees, the expected mean count is about 220.636, close to the measured 220.389. All rows are covered, with counts from 185 to 261. Early Brier values score only covered rows, so both the averaging ensemble and potentially the evaluated row set change as trees are added. The code independently recomputes all OOB averages from the stored prediction matrix and masks, and checks selected masks against direct membership in the bootstrap lists. The 600-tree validation Brier is about 0.155869, versus OOB 0.157494. Different evaluation rows and different effective ensembles explain why they need not agree. Furthermore, settings were selected using training CV, so this OOB score is an internal diagnostic, not a wholly independent assessment of the entire selection procedure.

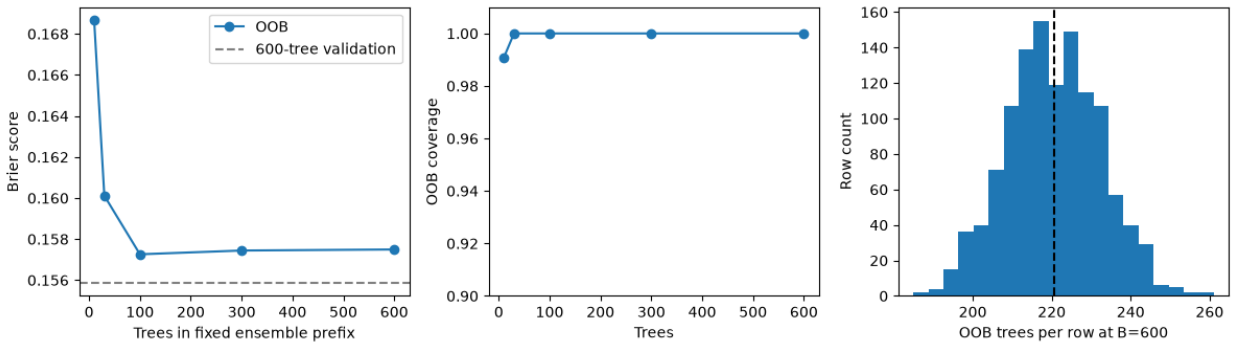


Figure 1: OOB error, coverage, and eligible-tree counts. More trees stabilize averaging but do not guarantee a strictly decreasing measured error curve.

6 Study variability at fixed inputs separately

Resample the training set ten times and fit a single tree and a full 300-tree forest to each resample, keeping selected m and leaf size fixed. Evaluate three fixed inputs with (x_1, x_2) equal, specified x_3, binary $x_4 = 0$, and remaining coordinates zero. At the middle profile $(x_1, x_2, x_3, x_4) = (0, 0, 0.2, 0)$, the tree probability standard deviation is about 0.3544, compared with 0.1309 for the forest. At the low profile the values are about 0.0416 and 0.0086. At the high profile the tree always predicts one in these ten resamples and has zero observed variability, while the forest varies slightly. These are descriptive bootstrap results at fixed inputs, unlike the earlier correlations across inputs. Ten resamples do not give precise variance estimates, and settings were not reselected. Averaging often helps stability, but the data do not support claiming a lower observed variance at every input.

7 Compare training MDI with validation permutation importance

For each split on feature j, impurity decrease is parent impurity minus the child-size-weighted impurity average. Weight by the fraction of training mass reaching that node, sum across nodes using j, and normalize to obtain training MDI. In a forest, `feature_importances_` aggregates tree importances. This credits reductions achieved while fitting training labels; it is not a held-out prediction score or a causal effect.

Permutation importance instead keeps the fitted model unchanged, shuffles one validation feature, and measures

$$\Delta_j = \text{Brier}(y, \hat{p}(X_{\text{shuffle}(j)})) - \text{Brier}(y, \hat{p}(X)).$$

Restore the original matrix for each shuffle and repeat ten times. Positive change means shuffling hurts. Negative small values can arise from noise. Shuffle standard deviation describes permutation randomness, not population uncertainty.

Feature	Training MDI	Mean validation Brier increase
x_1 (signal)	0.3059	0.0316
x_2 (near-copy)	0.2839	0.0268
x_3 (signal)	0.2114	0.0512
x_4 (binary noise)	0.0017	0.0002

Continuous noise features receive nonzero training credit because their many possible thresholds can exploit chance patterns. Validation generally supports far less usefulness. MDI and permutation values have different scales and should not be numerically equated.

Jointly permute (x_1, x_2) using the same row permutation, preserving their near-equality while breaking their relationship with the target and remaining inputs. The joint Brier increase is about 0.1314, much larger than the sum of individual increases. Individual shuffles leave some redundant information available and create atypical feature combinations. The scores are not additive allocations of intrinsic causal importance. The validation diagnostics do not trigger feature

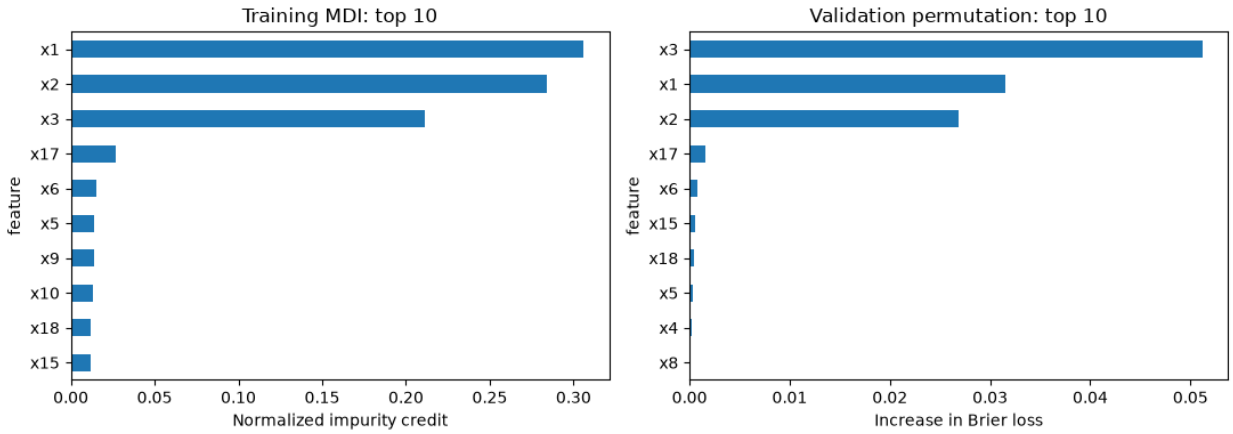


Figure 2: Training split credit and validation permutation effects answer different questions. Redundancy and chance split opportunities affect the apparent rankings.

deletion or model retuning within this experiment.

8 Final test comparison

Compare the prevalence baseline, single tree, 300-tree bagging, and selected 300-tree forest. The separate manual 600-tree diagnostic does not replace the prespecified bagging baseline.

Model	Brier	Clipped log loss	Trees	Total nodes
Prevalence	0.25000	0.69315	0	0
Single tree	0.20461	1.71822	1	75
Bagging ($m = 20$)	0.18481	0.55925	300	15000
Selected forest ($m = 10$)	0.18281	0.54847	300	15656

Averaging improves probability quality substantially relative to one tree here. The selected forest is only slightly better than bagging on this test; the result does not establish a universal ranking. High single-tree log loss highlights occasional confident mistakes. Node count is a rough storage proxy, not measured memory consumption. Recorded training times come from the original fits; prediction time is the median of ten calls on the same 400-row feature batch. One-worker execution improves local comparability, but timings are not portable guarantees.

Resolution of the interpretation questions

Smaller m trades stronger randomization against weaker feature choices, so improvement is not monotone. OOB predictions must exclude trees that trained on the scored row and handle zero coverage explicitly. OOB and validation metrics evaluate different samples. Correlations across input vectors differ from fixed-input fitting variability. Redundant features defeat simple additive importance readings, and importance is not causality. Finally, labels remain stochastic: good probabilities do not imply perfect classification.

Further explanation: what averaging and OOB can establish

Probability averaging is not majority voting

Consider three tree probabilities $(0.51, 0.51, 0.01)$ for the same input. Two trees would individually vote positive at threshold 0.5, yet their mean probability is about 0.343, which gives a negative decision after averaging. The notebook uses probability averaging throughout. Confusing these procedures changes both probability metrics and decisions, even though both are informally called forest voting.

Derive the variance expression with its assumptions

For equal-variance tree predictions $T_b(x)$ at one fixed input, with common variance σ^2 and common pairwise correlation ρ,

$$\operatorname{Var}\left(B^{-1}\sum_b T_b(x)\right) = B^{-2}\{B\sigma^2 + B(B-1)\rho\sigma^2\}.$$

The first part comes from the B individual variances and the second from all distinct ordered covariance pairs. This yields the expression in Section 3. More trees suppress the $1/B$ component but do not eliminate the correlated component under these assumptions. In practice tree variances and correlations need not all be equal; the equation is an explanatory model, not an exact law for every fitted forest.

Correlation across 400 different inputs answers a different question. Trees that follow the same genuine signal can have high across-input correlation even when their fitting errors are partly independent. One must not insert that empirical vector-correlation statistic directly into the fixed-input variance formula and call the result a sampling variance estimate.

An OOB score evaluates a smaller ensemble per row

At 600 trees each row uses about 221 OOB trees, whereas a new input can use all 600. OOB is not exactly equivalent to applying the full forest to an independent set, even with impeccable book-keeping. The bootstrap base learners also see repeated rows rather than an ordinary independent sample of 1,200 distinct cases. These differences help explain discrepancies between OOB and validation scores without automatically indicating an error.

The expected number of distinct included rows in one bootstrap is

$$n\{1 - (1 - 1/n)^n\} \approx 0.632n.$$

This is different from the number of draws, which is exactly n. The distinction matters when implementing minimum leaf sizes through sample weights. It also explains why a small number of trees may leave some rows without any eligible OOB prediction.

Selection and preprocessing can leak into an OOB claim

A tree's bootstrap exclusion protects a row only from that tree's direct fit. If a preprocessing transformation was first estimated using all training targets, an excluded row may still influence the fitted representation. This notebook avoids that issue by using no learned preprocessing before bootstrapping. Likewise, choosing hyperparameters based on the same training records means an OOB score is not a fresh assessment of the entire selection procedure. The independent test is still needed. If features were to be removed after the validation importance study, that would create a new selection step, not a free improvement with the original validation evidence remaining untouched.

Importance depends on the question being asked

Shuffling x_1 alone breaks its relationship with x_2 and may create combinations almost absent from training. The measured loss increase mixes removal of information with response to this distribution change. Shuffling the pair jointly preserves their near-equality and removes their shared information together. The different result is expected; there is no rule requiring individual importance values to sum to the group's value. Neither procedure answers what would happen after retraining a model without the feature, which is another experiment.

Documentation details and practical tips

The RandomForestClassifier reference explains bootstrap sampling, feature subsampling, probability prediction, and OOB attributes. Its default OOB score, when requested in the usual way, is an accuracy score; the notebook deliberately computes an OOB Brier score from probabilities instead. Do not compare those numbers as if they were the same metric. `n_estimators` changes the number of trees, whereas `max_features` changes the features considered within a split.

Practical checks. Preserve class alignment for every tree, especially in small or imbalanced samples; report OOB coverage alongside loss; retain enough bootstrap bookkeeping to independently verify averages; and compare permutation measures on held-out rows without refitting. Increasing tree count reduces Monte Carlo variation but cannot compensate for an inappropriate feature set, unsuitable leaf size, or a changed data population.